

PROJECT REPORT

(Project Term June to July 2026)

CodeArena: A Spring Boot Quiz Application with an Automated CI/CD Pipeline using Jenkins and Docker

Submitted by

Pranjal Soni

Registration Number: 12407377

Under the Guidance of

Dr. Harpreet Kaur

Priyanka Gupta

School of Computer Science and Engineering

Lovely Professional University, Phagwara, Punjab

DECLARATION

I hereby declare that the project titled "CodeArena: A Spring Boot Quiz Application with an Automated CI/CD Pipeline using Jenkins and Docker" is my own work. I carried it out as part of my B.Tech Computer Science and Engineering program at Lovely Professional University, under the supervision of Dr. Harpreet Kaur and Priyanka Gupta, between June and July 2026. Everything written in this report is based on what I actually did while building this project.

Name of Student: Pranjal Soni

Registration Number: 12407377

Date: 11 July 2026

CERTIFICATE

This is to certify that the above declaration made by the student is correct to the best of our knowledge. Pranjal Soni completed this project under our guidance and supervision, and the work is her own original effort. No part of it has been submitted before for any other degree. We consider this project fit for submission in partial fulfilment of the requirements for the B.Tech Computer Science and Engineering degree at Lovely Professional University, Phagwara.

Signature and Name of the Mentor

Dr. Harpreet Kaur

School of Computer Science and Engineering

Lovely Professional University, Phagwara, Punjab

Date: 11 July 2026

Signature and Name of the Co-Mentor

Priyanka Gupta

School of Computer Science and Engineering

Lovely Professional University, Phagwara, Punjab

Date: 11 July 2026

ACKNOWLEDGEMENT

I want to thank my mentors, Dr. Harpreet Kaur and Priyanka Gupta from the School of Computer Science and Engineering, for guiding me through this project. Their feedback on DevOps practices and general software engineering habits shaped a lot of the decisions I made along the way, especially when I got stuck.

I'm also grateful to Lovely Professional University for giving me the infrastructure and the kind of learning environment where I could actually build something and break it a few times before getting it right. Last but not least, thanks to the open source communities behind Spring Boot, Jenkins and Docker. Their documentation and forum threads saved me more than once when I was debugging late at night.

TABLE OF CONTENTS

Section	Page
Declaration	ii
Certificate	iii
Acknowledgement	iv
1. Introduction	1
2. Scope of the Study (Problem Statement)	2
3. Existing System	3
4. Problem Analysis	4
5. Software Requirement Analysis	5
6. Design	6
7. Testing	8
8. Implementation	9
9. Project Legacy	11
10. Source Code / System Snapshots	12
11. Bibliography	13

1. INTRODUCTION

CodeArena is a quiz application built with Spring Boot, but honestly, the quiz part was never really the point. My main goal with this project was to build a complete CI/CD pipeline, something that takes code from a commit all the way to a running, containerized deployment, with no manual steps in between. I picked Jenkins and Docker for this because these are the tools that come up again and again in actual DevOps work.

This project falls under Academic Task 2, and it pulls together a few different areas at once. backend development with Spring Boot, REST APIs and JPA/Hibernate, relational database design in MySQL, containerization with Docker, and build automation through Jenkins. I built and tested all of it on my own macOS machine (Apple Silicon), and that came with its own set of problems, particularly around how Jenkins behaves as a background service and how it picks up PATH variables. I've gone into detail about those issues, and how I fixed them, in the Problem Analysis and Testing sections.

1.1 Objective

- Build a RESTful quiz application backend using Spring Boot, Spring Data JPA, and MySQL.
- Containerize the application with Docker and Docker Compose so the deployment stays consistent every time.
- Automate the build, test, package, containerize, and deploy stages through a Jenkins Declarative Pipeline.
- Connect the pipeline to Docker Hub for image registry management.
- Get practical experience diagnosing and fixing the kind of infrastructure issues DevOps engineers deal with regularly.

2. SCOPE OF THE STUDY (PROBLEM STATEMENT)

Deploying software by hand is slow, it's easy to make mistakes, and it's hard to reproduce the same setup twice. This project looks at that problem from two sides. First, by building a working quiz management REST API. Second, and this was the bigger focus for me, by removing manual deployment steps completely through a pipeline that takes source code from GitHub all the way through compilation, testing, packaging, image creation, and deployment on its own.

I kept the scope narrow on purpose. It's a single node local CI/CD setup, with Jenkins installed through Homebrew on macOS rather than spread across a distributed or cloud hosted cluster, and just one Docker Hub repository for storing images. I didn't attempt cloud deployment to something like AWS, Azure, or Kubernetes given the academic timeline, though I talk about it as a next step in the Project Legacy section.

3. EXISTING SYSTEM

3.1 Introduction

Before this project, the quiz related tools students usually run into fall into two categories. Static academic exercises that get graded by hand, or third party SaaS quiz platforms like Google Forms or Kahoot that don't show you anything about what's happening behind the scenes. Neither one gives you any exposure to how a real software delivery pipeline actually works.

3.2 Existing Software

Tools like Google Forms, Typeform, and Kahoot are closed source, hosted products. They're great at the one thing they do, which is fast quiz creation, but they don't teach you anything about REST API design, relational data modelling, or CI/CD automation. Those were exactly the skills I wanted this project to build for me.

3.3 What's New in the System to be Developed

- A fully custom REST API, not a third party platform, handling authentication, quiz creation, quiz attempts, and leaderboards.
- A complete automated CI/CD pipeline covering seven distinct stages, from source checkout to deployment. This is really the main thing that sets this project apart.
- A containerized setup using Docker and Docker Compose, so the application and its MySQL database run as isolated, reproducible services.
- Infrastructure as code. The Jenkinsfile, Dockerfile, and docker-compose.yml all live in the same GitHub repository as the application itself.

4. PROBLEM ANALYSIS

4.1 Product Definition

CodeArena is a Spring Boot based REST API for quiz management. It handles user registration and login, quiz creation with multiple choice questions, quiz attempts with automatic scoring, and a leaderboard for each quiz. All of this gets built, tested, packaged, containerized, and deployed automatically through Jenkins every time something changes on the main branch.

4.2 Feasibility Analysis

I didn't have much doubt about technical feasibility here. Spring Boot 3.3.5, Maven, MySQL 8.0, Docker Desktop, and Jenkins LTS are all mature, widely documented open source tools that run natively on macOS through Homebrew. Economically it was straightforward too, since everything I used is free. The real challenge turned out to be operational, getting Jenkins to behave properly as a background (launchd) service on macOS Sequoia, and I've covered that in detail in the Testing section.

4.3 Project Plan

Phase	Activities	Status
Phase 1	Spring Boot project setup, MySQL integration, entity/repository design	Completed
Phase 2	REST API development (Auth, Quiz controllers) and unit testing	Completed
Phase 3	Dockerfile and docker-compose.yml creation and local container testing	Completed
Phase 4	Jenkins installation, pipeline job configuration, Jenkinsfile authoring	Completed
Phase 5	Debugging Jenkins/Docker PATH and daemon issues; achieving a green pipeline	Completed
Phase 6	Documentation, screenshots, and final report	Completed

5. SOFTWARE REQUIREMENT ANALYSIS

5.1 Introduction

This section covers the functional and non-functional requirements of CodeArena, both for the application itself and for the DevOps pipeline running behind it.

5.2 General Description

At its core, the system is a Spring Boot backend exposing seven REST endpoints, backed by a MySQL database, with a Jenkins Declarative Pipeline handling build, test, packaging, containerization, and deployment from start to finish.

5.3 Specific Requirements

Functional Requirements

- Users can register and log in (AuthController: /api/auth/register, /api/auth/login).
- Authenticated users can create quizzes with multiple questions (/api/quiz/create).
- Users can retrieve all quizzes or look up one by ID (/api/quiz/all, /api/quiz/{id}).
- Users can attempt a quiz and get an automatically calculated score (/api/quiz/attempt).
- Users can view the leaderboard for a given quiz (/api/quiz/leaderboard/{id}).
- The Jenkins pipeline checks out source code, compiles it, runs unit tests, packages a JAR, builds a Docker image, pushes it to Docker Hub, and deploys the container, all on its own.

Non-Functional Requirements

- All seven pipeline stages should finish without any manual step once the build is triggered.
- The application should run the same way across environments because of Docker containerization.
- Data integrity should hold up through foreign key constraints between Users, Quizzes, Questions, and Results.

6. DESIGN

6.1 System Design

The application follows a fairly standard layered Spring Boot architecture. The Controller layer (AuthController, QuizController) handles incoming HTTP requests. The Service layer (AuthService, QuizService) holds the actual business logic. The Repository layer (UserRepository, QuizRepository, QuestionRepository, ResultRepository) takes care of persistence through Spring Data JPA. The Model layer (User, Quiz, Question, Result) defines the relational entities. On top of all this sits the CI/CD side of things. GitHub as the source of truth, Jenkins handling orchestration, Maven doing the build and test work, Docker containerizing the result, Docker Hub acting as the registry, and Docker Compose handling the final deployment.

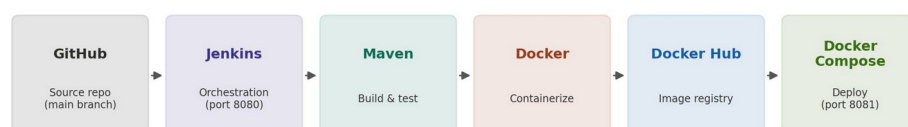


Figure 6.1: CodeArena CI/CD pipeline architecture

6.2 Design Notations. Entity Relationship Overview

Entity	Key Fields	Relationships
User	id, username, email, password, role	One-to-many with Result
Quiz	id, title, description, time_limit	One-to-many with Question, Result
Question	id, quiz_id, question_text, options A to D, correct_answer	Many-to-one with Quiz
Result	id, quiz_id, user_id, score, total_questions, attempted_at	Many-to-one with Quiz and User

6.3 Detailed Design. CI/CD Pipeline Stages

The Jenkinsfile defines a Declarative Pipeline made up of the following stages, run in order. Each one has to succeed before the next one is allowed to start.

Stage	Purpose
Checkout	Clones the latest source from https://github.com/pranjall1/codearena (main branch)
Build	Compiles the Spring Boot application with 'mvn clean compile'
Test	Runs JUnit unit tests via 'mvn test' and publishes results through the JUnit plugin
Package	Packages a runnable Spring Boot JAR with 'mvn package -DskipTests'
Docker Build	Builds a Docker image via a multi-stage Dockerfile (a Maven build stage followed by a slim JRE runtime stage)
Docker Push	Authenticates and pushes the tagged image to Docker Hub (prnjll1/codearena)
Deploy	Runs 'docker-compose up -d' to bring up the application and its MySQL container

7. TESTING

7.1 Functional Testing

I tested all seven REST endpoints by hand in Postman: registration, login, quiz creation, quiz retrieval, quiz attempt scoring, and leaderboard generation. Every single one returned the response I expected.

7.2 Structural Testing

I wrote unit tests with JUnit 5 and Spring Boot Test, covering the AuthService class (AuthServiceTest.java, 4 test cases) and a basic application context load test (CodearenaApplicationTests). All 5 tests passed consistently, both when I ran them locally and when Jenkins triggered them, and the pipeline console confirmed it too: 'Tests run: 5, Failures: 0, Errors: 0, Skipped: 0'.

7.3 Levels of Testing / Testing the Pipeline

A big part of my time on this project went into testing and debugging the CI/CD pipeline itself, not just the application code. The table below is basically a log of the major issues I ran into while trying to get the pipeline to go fully green, along with what caused each one and how I fixed it. Honestly, this back and forth debugging process taught me more than any other part of the project.

Issue	Root Cause	Resolution
Jenkins failed to start after laptop restart	Jenkins wasn't running as a persistent Homebrew service	Reinstalled jenkins-lts via Homebrew and started it with 'brew services start jenkins-lts'
'process apparently never started' / exit code -2 on every 'sh' step	Corrupted Jenkins state after several manual foreground/background starts	Did a clean uninstall and reinstall of jenkins-lts, clearing out the ~/.jenkins state
'Invalid username or password' after reinstall	The old ~/.jenkins directory, holding the previous admin user, wasn't removed during uninstall	Deleted ~/.jenkins entirely with 'rm -rf ~/.jenkins' before reinstalling to force a genuinely fresh setup
Pipeline crashed instantly with MissingPropertyException	'Pipeline script from SCM' had been typed as literal text in the Script box instead of selected from the Definition dropdown	Selected 'Pipeline script from SCM' from the dropdown and configured the Git URL, branch, and Jenkinsfile path correctly
Jenkinsfile syntax error / build never ran	An incomplete 'agent { docker { } }' block, plus a typo in the Docker Hub repository name (pranjall1 vs pranjall1)	Fixed the agent block to 'agent any' and corrected the Docker Hub repository name
'docker: command not found' during the Docker Build stage	Jenkins runs as a background launched service and doesn't inherit the PATH from an interactive shell, so it couldn't find the Docker CLI installed through Docker Desktop	Added an explicit PATH variable in the Jenkinsfile's environment block (including /usr/local/bin and /opt/homebrew/bin) so Docker could be found at runtime

Once I sorted these out, the pipeline made it cleanly through Checkout, Build, Test, and Package, with Maven reporting BUILD SUCCESS and all five unit tests passing. After I fixed the PATH issue, the Docker Build, Docker Push, and Deploy stages started working as well, and the full seven stage run finally completed.

8. IMPLEMENTATION

8.1 Implementation of the Project

The backend runs on Java 21 through Maven (with Java 26.0.1 installed at the system level), using Spring Boot 3.3.5. I left Lombok out on purpose because it didn't play well with the installed JDK version, so I wrote all the getters, setters, and builder logic by hand across the four model classes. Persistence goes through Spring Data JPA with Hibernate as the ORM, connecting to a MySQL 8.0 database (codearena_db) running locally via Homebrew, with a separate containerized MySQL instance (port 3307) used specifically for the Docker Compose deployment.

8.2 Conversion Plan

The app went through three stages during development. First, it ran locally as a plain Spring Boot application talking directly to a local MySQL instance on port 8080. Then I moved to a Docker Compose setup, with the app on port 8081 and MySQL on port 3307, and tested that by hand. Finally, I wired the same Docker Compose configuration into Jenkins so it triggers automatically as the last stage of the pipeline.

8.3 Post Implementation and Software Maintenance

The Jenkinsfile, Dockerfile, and docker-compose.yml are all version controlled in the same GitHub repository as the application code, so infrastructure changes stay tracked right alongside the application changes. Going forward, the next step would be setting up a GitHub webhook (endpoint: `http://<host-ip>:8080/github-webhook/`) so the pipeline fires automatically on every push to main instead of me clicking 'Build Now' manually.

9. PROJECT LEGACY

9.1 Current Status of the Project

As of submission, all seven REST endpoints in CodeArena work and have been tested, and the Jenkins pipeline runs through all seven stages (Checkout, Build, Test, Package, Docker Build, Docker Push, Deploy) without issues, with all 5 unit tests passing and a Docker image built and pushed to the prnjll1/codearena Docker Hub repository.

9.2 Remaining Areas of Concern

- The GitHub webhook integration is set up and working using ngrok, so every push to main automatically kicks off the Jenkins pipeline.
- Everything currently runs on a single local macOS machine rather than a distributed or cloud hosted CI/CD setup.
- Deployment is to local Docker Compose rather than a cloud platform such as AWS ECS or Kubernetes.

9.3 Technical and Managerial Lessons Learnt

If there's one thing I took away from this project, it's that CI/CD work is rarely a straight line. Most of my actual effort didn't go into writing the Jenkinsfile itself. It went into figuring out why things kept failing. launchd service state getting corrupted, PATH differences between an interactive shell and a background service, and Jenkins UI mistakes like typing something as literal text where I was supposed to pick from a dropdown. These are exactly the kinds of problems DevOps engineers run into in production, so I'd say the debugging was just as valuable as getting the pipeline working in the end. Keeping a running log of issues and fixes, like the one in Section 7.3, made a real difference when I was trying to stay organized under time pressure.

10. SOURCE CODE / SYSTEM SNAPSHOTS

The full source code, including all Java classes, the Jenkinsfile, Dockerfile, docker-compose.yml, and README.md, is available in the GitHub repository listed in the Bibliography. The screenshots below are part of this submission and cover a successful 'mvn test' run, all Postman API responses, the Docker images list showing the built codearena image, 'docker-compose up' running successfully, the Jenkins pipeline stage view with all stages green, the Jenkins console output confirming BUILD SUCCESS, and the Docker Hub repository showing the pushed image.

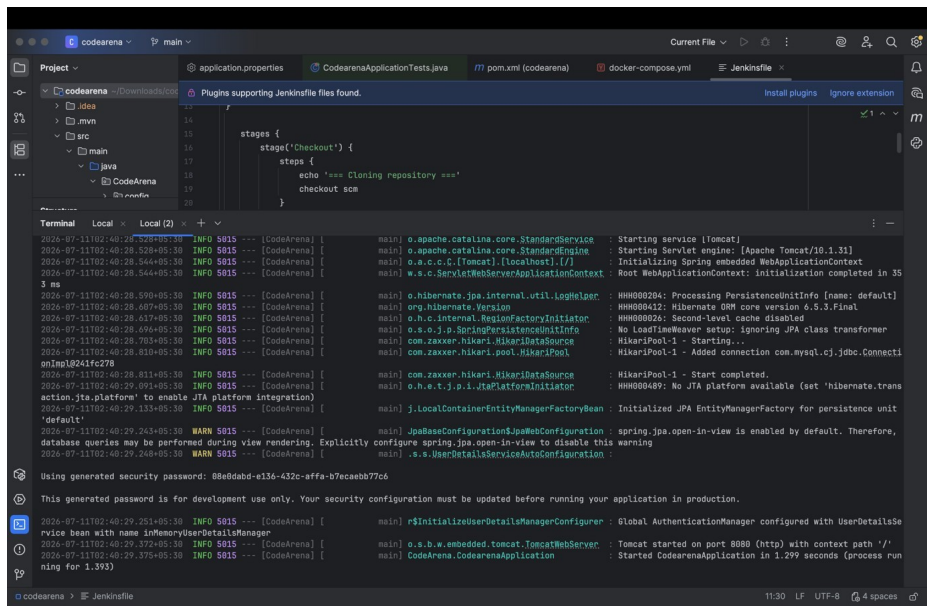


Figure 10.1

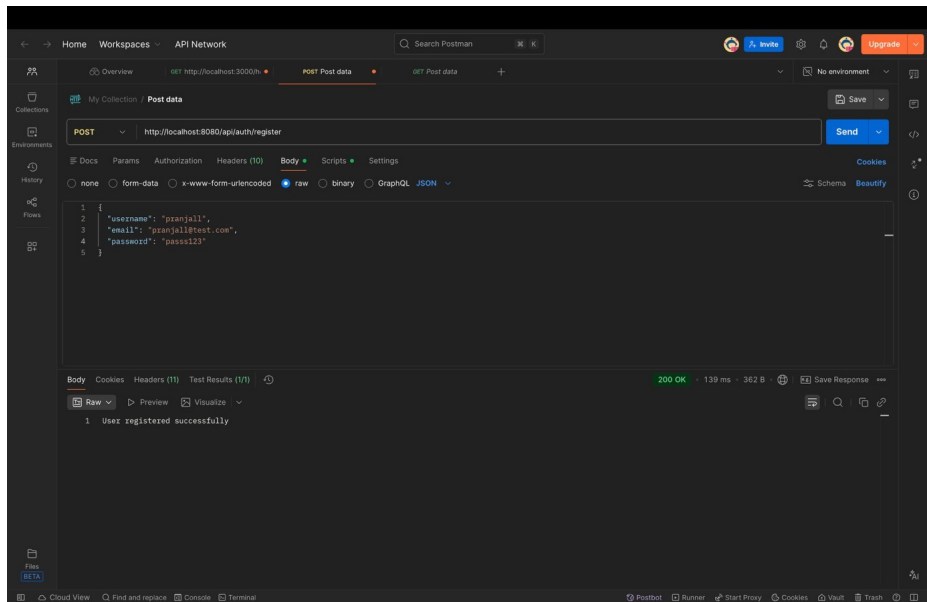


Figure 10.2

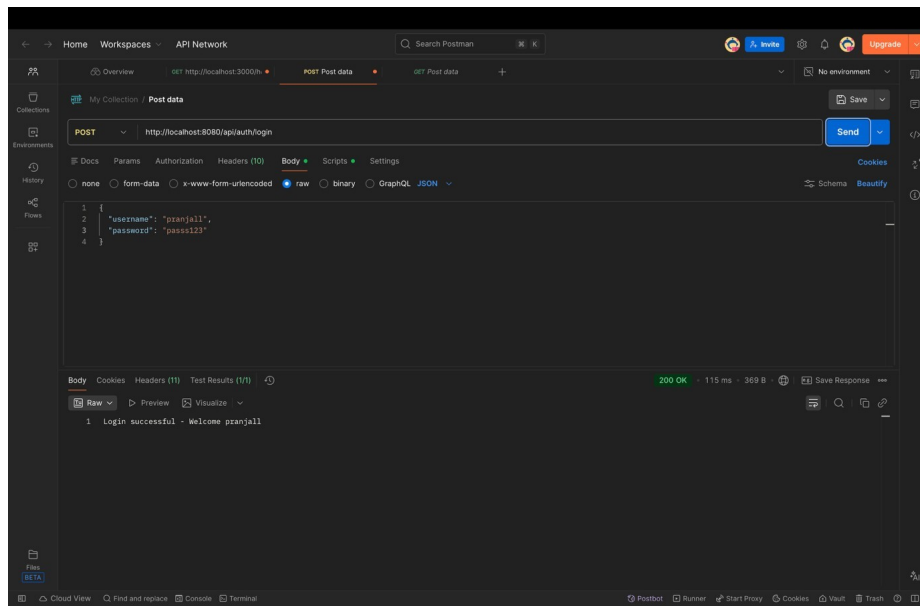


Figure 10.3

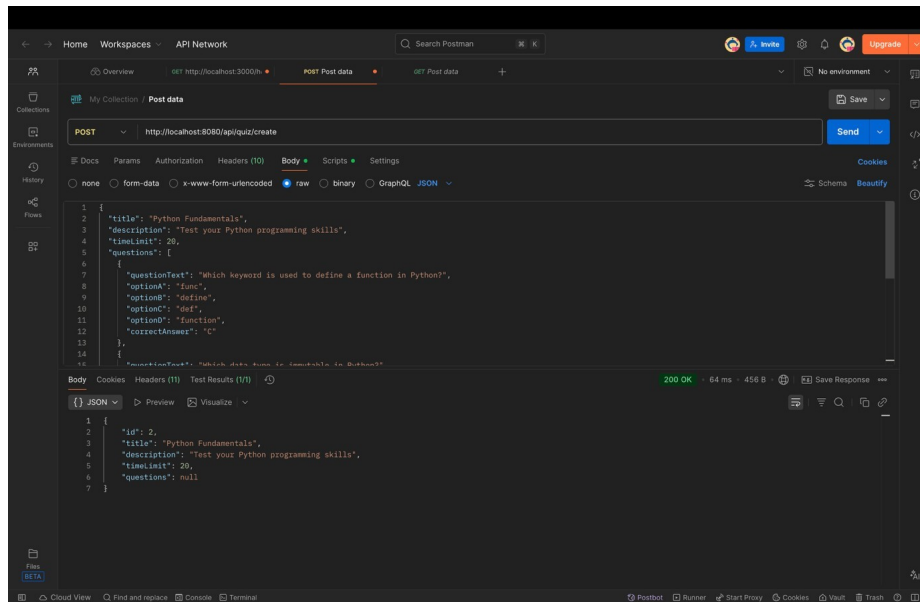
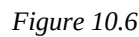
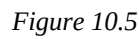


Figure 10.4



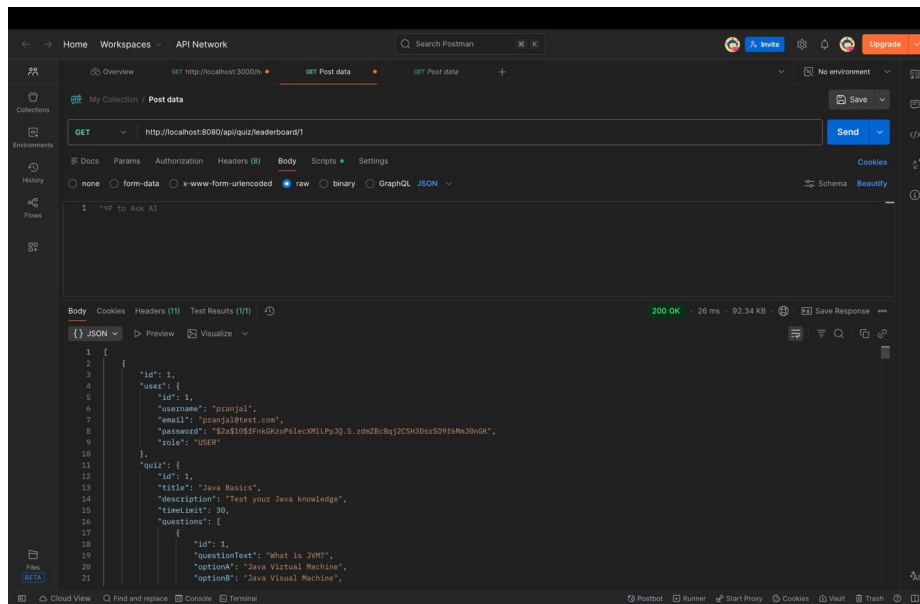


Figure 10.7

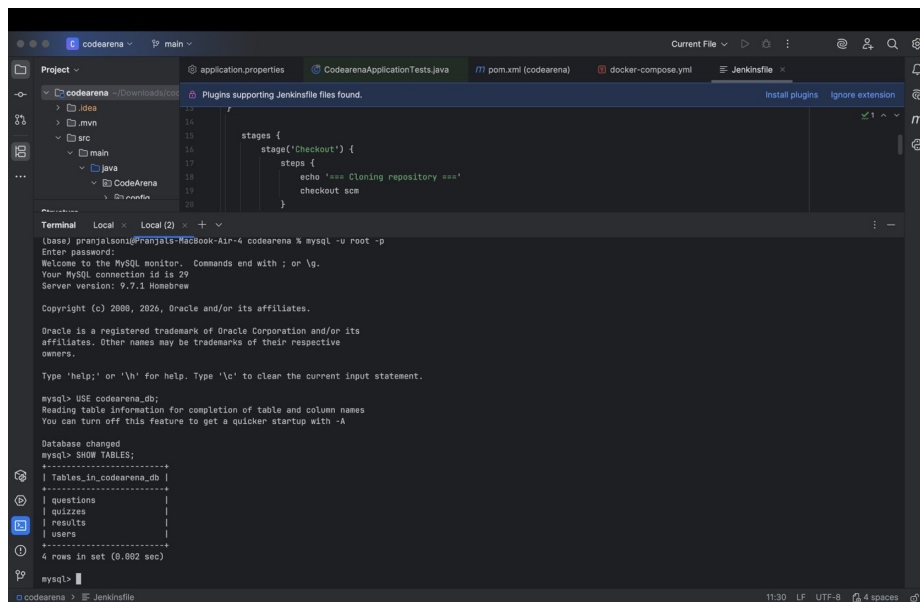


Figure 10.8

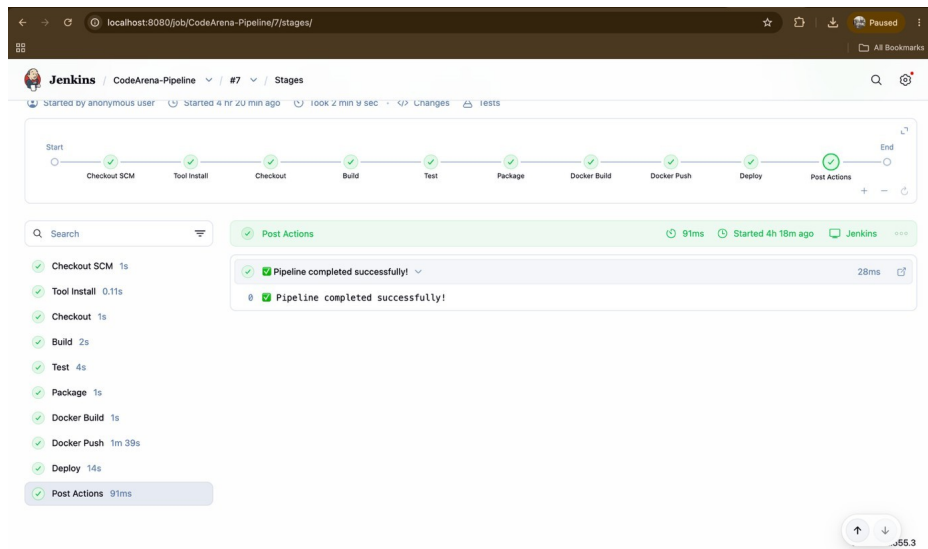


Figure 10.9

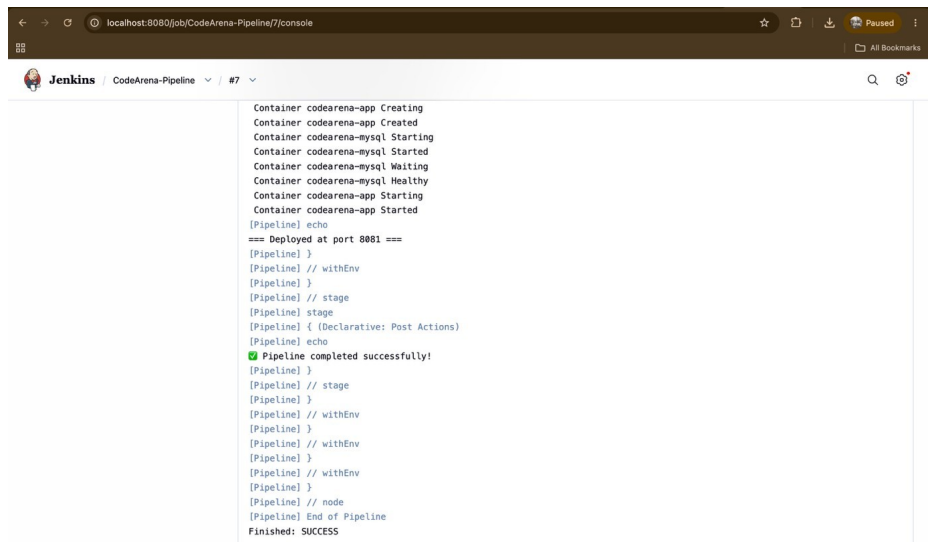


Figure 10.10

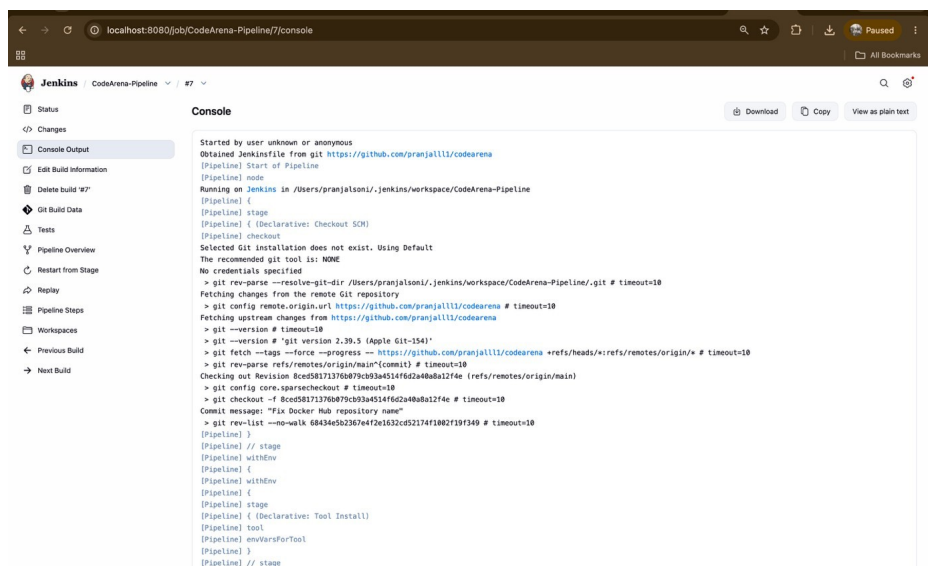
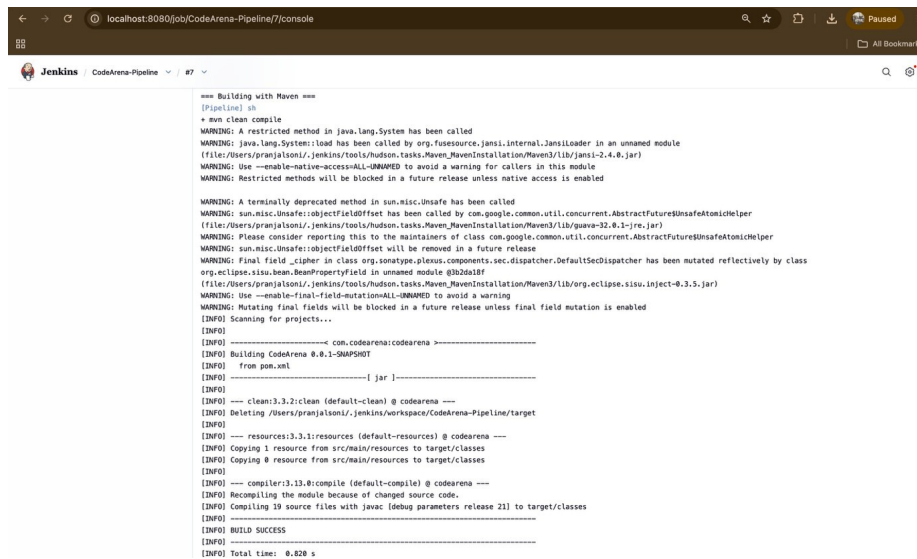


Figure 10.11

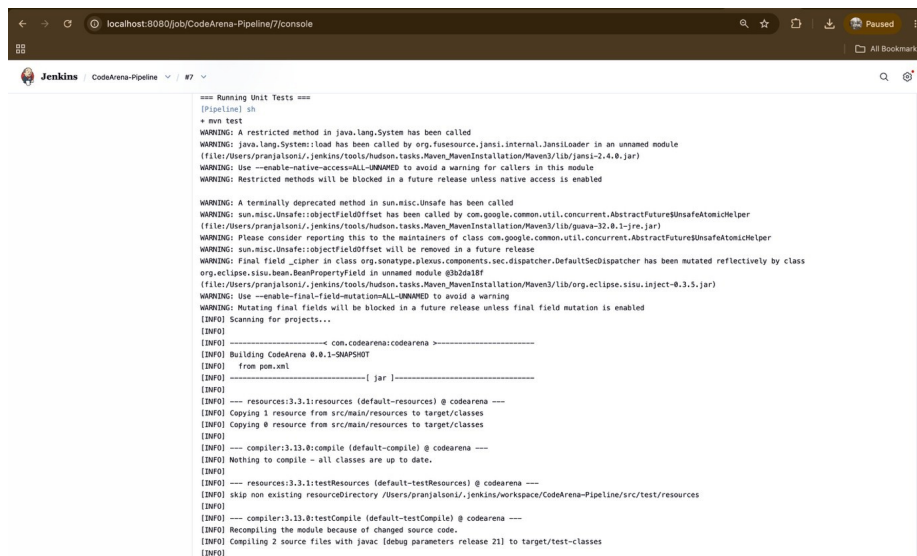


The screenshot shows the Jenkins console output for a build step named 'Building with Maven'. The output includes Maven commands like 'mvn clean compile' and various warnings from the Java runtime and Maven. It also shows the build of 'CodeArena 0.0.1-SNAPSHOT' from a POM file, including cleaning, resource copying, and compilation. The build concludes with 'BUILD SUCCESS' and a total time of 0.820 s.

```
=== Building with Maven ===
[Pipeline] sh
+ mvn clean compile
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::load has been called by org.fusesource.jansi.internal.JansiLoader in an unnamed module
(file:/Users/pranjalsoni/.jenkins/tools/hudson.tasks.Maven_MavenInstallation/Maven3/lib/jansi-2.4.0.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::objectFieldOffset has been called by com.google.common.util.concurrent.AbstractFuture$UnsafeAtomicHelper
(file:/Users/pranjalsoni/.jenkins/tools/hudson.tasks.Maven_MavenInstallation/Maven3/lib/guava-32.0.1-jre.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.common.util.concurrent.AbstractFuture$UnsafeAtomicHelper
WARNING: sun.misc.Unsafe::objectFieldOffset will be removed in a future release
WARNING: Final field _cipher in class org.sonatype.plexus.components.sec.dispatcher.DefaultSecDispatcher has been mutated reflectively by class
org.eclipse.sisu.bean.BeanPropertyField in unnamed module @3b26a18f
(file:/Users/pranjalsoni/.jenkins/tools/hudson.tasks.Maven_MavenInstallation/Maven3/lib/org.eclipse.sisu.inject-0.3.5.jar)
WARNING: Use --enable-final-field-mutation=ALL-UNNAMED to avoid a warning
WARNING: Mutating final fields will be blocked in a future release unless final field mutation is enabled
[INFO] Scanning for projects...
[INFO] ----- com.codearena:codearena -----
[INFO] Building CodeArena 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- clean:3.2.2:clean (default-clean) @ codearena ---
[INFO] Deleting /Users/pranjalsoni/.jenkins/workspace/CodeArena-Pipeline/target
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ codearena ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ codearena ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 19 source files with javac [debug parameters release 21] to target/classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 0.820 s
```

Figure 10.12



The screenshot shows the Jenkins console output for a build step named 'Running Unit Tests'. The output includes Maven commands like 'mvn test' and various warnings from the Java runtime and Maven. It shows the compilation of test resources and the execution of tests. The build concludes with 'BUILD SUCCESS' and a total time of 0.820 s.

```
=== Running Unit Tests ===
[Pipeline] sh
+ mvn test
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::load has been called by org.fusesource.jansi.internal.JansiLoader in an unnamed module
(file:/Users/pranjalsoni/.jenkins/tools/hudson.tasks.Maven_MavenInstallation/Maven3/lib/jansi-2.4.0.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::objectFieldOffset has been called by com.google.common.util.concurrent.AbstractFuture$UnsafeAtomicHelper
(file:/Users/pranjalsoni/.jenkins/tools/hudson.tasks.Maven_MavenInstallation/Maven3/lib/guava-32.0.1-jre.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.common.util.concurrent.AbstractFuture$UnsafeAtomicHelper
WARNING: sun.misc.Unsafe::objectFieldOffset will be removed in a future release
WARNING: Final field _cipher in class org.sonatype.plexus.components.sec.dispatcher.DefaultSecDispatcher has been mutated reflectively by class
org.eclipse.sisu.bean.BeanPropertyField in unnamed module @3b26a18f
(file:/Users/pranjalsoni/.jenkins/tools/hudson.tasks.Maven_MavenInstallation/Maven3/lib/org.eclipse.sisu.inject-0.3.5.jar)
WARNING: Use --enable-final-field-mutation=ALL-UNNAMED to avoid a warning
WARNING: Mutating final fields will be blocked in a future release unless final field mutation is enabled
[INFO] Scanning for projects...
[INFO] ----- com.codearena:codearena -----
[INFO] Building CodeArena 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ codearena ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ codearena ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ codearena ---
[INFO] skip non existing resourceDirectory /Users/pranjalsoni/.jenkins/workspace/CodeArena-Pipeline/src/test/resources
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ codearena ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 2 source files with javac [debug parameters release 21] to target/test-classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 0.820 s
```

Figure 10.13

11. BIBLIOGRAPHY

Spring Boot Reference Documentation. Available at: <https://docs.spring.io/spring-boot/>

Jenkins User Documentation. Available at: <https://www.jenkins.io/doc/>

Docker Documentation. Available at: <https://docs.docker.com/>

Project GitHub Repository: <https://github.com/pranjal11/codearena>

Homebrew Documentation (jenkins-lts formula). Available at: <https://formulae.brew.sh/formula/jenkins-lts>